# PowerUp Session 02 — The Other Side of `fetch`: The B4F Hub Server

## Session goal

Last Saturday we worked on the **client side** (React). Every time React needed data it called something like `fetch("/api/opportunities")`. Today we opened **the other side of that request**: the B4F Hub server — which program receives the request, which code answers it, where the data comes from, and how the answer travels back to React.

Everything in this guide matches the code in your repository's `server/` folder — the exact server we read together in class.

## The full picture

TEXT

```
Browser — React app (port 5173)
   │  fetch("/api/opportunities")        ← client/src/api.ts
   ▼
HTTP request: GET /api/opportunities
   │  Vite forwards every /api request to port 3001
   ▼
Node.js process (port 3001)               ← npm start → node index.js
   ▼
Express app                               ← server/index.js
   │  app.get("/api/opportunities", ...) matches
   ▼
Route handler (req, res) => { ... }       ← server/index.js
   │  reads the opportunities array in memory
   ▼
res.json(opportunities)  →  HTTP response: 200 + JSON body
   ▼
React: response.json() → useState → cards on screen
```

## What is Node.js?

- **JavaScript is a language.** The browser is one program that can run it. **Node.js** is another program that runs JavaScript files directly, with no browser. Node.js is a **runtime** — not a new language, and not a framework.
- The browser gives JavaScript tools like `document`, `window` and `localStorage`. Node gives different tools — for example, listening on a network **port**. That is why React code cannot run inside the server, and server code cannot run in a browser tab.
- You already use Node every day: `npm install` and `npm run dev` are Node programs.

**A script vs a server.** `server/test.js` contains one line, `console.log("Hello world");`. Run it with `node test.js`: it prints and **finishes**. Run the server with `npm start`: it starts listening and **keeps running**, waiting for requests, until you press Ctrl+C. That is why the server needs its own terminal.

A **port** is a door number on your computer: the server listens on **3001**, the React dev server on **5173**.

## What is Express?

**Express** is an npm package (a framework) that runs on Node and makes writing a server simple:

- `const app = express()` creates the application;
- `app.get(path, handler)`, `app.post(path, handler)`, `app.patch(path, handler)` mean: "when a request with **this method** and **this path** arrives, run **this function**";
- every handler receives `req` (the request) and `res` (the response);
- `app.listen(PORT, ...)` starts the server.

## The server folder

TEXT

```
server/
├── package.json        "npm start" = "node index.js"; dependency: express
├── package-lock.json   exact installed versions
├── index.js            THE SERVER: creates the app, holds the data, every route, listens
├── data.js             starting data: 18 posts and 20 opportunities
└── test.js             a one-line script (script vs server demo)
```

At this stage **almost everything lives in one file: `index.js`.** That is normal for a small server, and it is the structure we read today.

## How the server starts — `index.js` from the top

JS

```
import express from "express";
import { initialPosts, initialOpportunities } from "./data.js";

const PORT = 3001;

const app = express();
app.use(express.json());

let posts = initialPosts;
let opportunities = initialOpportunities;
let nextPostId = Math.max(...posts.map((post) => post.id)) + 1;

const ALLOWED_CATEGORIES = ["announcement", "event", "community", "resource"];
const MIN_CONTENT_LENGTH = 3;
const MAX_CONTENT_LENGTH = 2000;

function delay(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}
```

1. **Imports** — Express, and the starting data from `data.js`.
2. **`PORT = 3001`** — the door number.
3. **`const app = express()`** — Express is created here. There is exactly one `app`.
4. **`app.use(express.json())`** — lets Express read a JSON request body and put it on `req.body`. Without this line, `req.body` would be empty for every `POST` and `PATCH`.
5. **The data** — `posts` and `opportunities` are plain arrays **in memory**. `nextPostId` is the id the next new post will get.
6. **Rules** — the allowed post categories and the content length limits.
7. **`delay(ms)`** — waits a little on purpose, so you can see React's loading state.

Then come the routes (next sections), and at the very end:

JS

```
app.listen(PORT, () => {
  console.log(`B4F Hub local API running at http://localhost:${PORT}`);
});
```

This is the line that actually **starts** the server.

## `req` and `res`

| You want | Use |
|---|---|
| a value from the URL, like `101` in `/api/opportunities/101` | `req.params.id` — always a **string** |
| the JSON the client sent in a `POST` / `PATCH` | `req.body` |
| send JSON back (status 200) | `res.json(data)` |
| send a different status | `res.status(404).json({ error: "..." })` |

`return res.status(...)...` sends the answer **and stops the function** — so a handler only ever sends one response.

## The endpoints that exist today

| Method | Path | What it does | Answers |
|---|---|---|---|
| GET | `/api/posts` | all posts, newest first | 200 |
| POST | `/api/posts` | create a post | 201, 400 |
| PATCH | `/api/posts/:id` | like / unlike a post | 200, 400, 404 |
| GET | `/api/opportunities` | all opportunities | 200 |
| PATCH | `/api/opportunities/:id` | apply to an opportunity | 200, 400, 404, 409 |

Any other path (for example `/api/opportunities/101` or `/api/health`) does not exist yet — Express answers **404** with a small HTML page saying `Cannot GET ...`.

### Status codes used by this server

| Code | Meaning | Real example |
|---|---|---|
| 200 | OK | `GET /api/opportunities` |
| 201 | Created something new | `POST /api/posts` |
| 400 | What you sent is invalid | `POST /api/posts` with content `"Hi"` |
| 404 | Nothing exists with that id | `PATCH /api/opportunities/999` |
| 409 | It exists, but this action conflicts with its current state | applying twice to the same opportunity |

## Request trace 1 — GET: loading the opportunities

JS

```
app.get("/api/opportunities", async (req, res) => {
  await delay(350);

  res.json(opportunities);
});
```

1. React calls `fetch("/api/opportunities")`; Vite forwards it to port 3001.
2. `express.json()` runs first (there is no body here).
3. `app.get("/api/opportunities", ...)` matches the method **GET** and the path.
4. The handler waits 350 ms, then `res.json(opportunities)` sends the array.
5. Response: **200** with 20 opportunities as JSON. React stores them in state and shows the cards.

## Request trace 2 — PATCH with `req.params` and `req.body`: Apply

React's Apply button sends `PATCH /api/opportunities/101` with the body `{ "applied": true }`.

JS

```
app.patch("/api/opportunities/:id", (req, res) => {
  const id = Number(req.params.id);
  const { applied } = req.body ?? {};

  if (applied !== true) {
    return res
      .status(400)
      .json({ error: "applied is required and must be true." });
  }

  const opportunity = opportunities.find((candidate) => candidate.id === id);

  if (!opportunity) {
    return res
      .status(404)
      .json({ error: `No opportunity found with id ${id}.` });
  }

  if (opportunity.applied) {
    return res
      .status(409)
      .json({ error: "You have already applied to this opportunity." });
  }

  opportunity.applied = true;

  res.json(opportunity);
});
```

- `:id` in the path is a **route parameter**: for `/api/opportunities/101`, `req.params.id` is the string `"101"`, so the code converts it with `Number(...)`.
- `req.body ?? {}` — if a request arrives with no body at all, this avoids a crash.
- The checks run **in order**, and each failure `return`s immediately: **400** (bad body) → **404** (no such id) → **409** (already applied).
- Otherwise it changes the opportunity **in memory** (`applied = true`) and answers **200** with the updated object.

## Request trace 3 — POST with `req.body`: publishing a post

React's post composer sends `POST /api/posts` with a body like `{ "content": "Hello B4F!", "category": "community" }`. The handler:

1. reads `content` and `category` from `req.body ?? {}`;
2. checks them, answering **400** for each problem: content must be a string, at least 3 characters (after trimming), at most 2000, and the category must be one of the four allowed values;

3. builds a new post with `id: nextPostId`, then increases `nextPostId`;
4. puts the new post at the front of the array: `posts = [newPost, ...posts];`
5. answers `res.status(201).json(newPost)` — **201 Created**.

**Why check on the server if the React form already checks?** Because `/api/posts` is a URL, not a page. Anyone can call it with curl or another program and skip the React form completely. Frontend checks give a nice user experience; **backend checks protect the data.**

## Request trace 4 — PATCH: like / unlike a post

`PATCH /api/posts/1` with `{ "liked": true }`: **400** if `liked` is not a boolean, **404** if there is no post with that id; otherwise the post is liked or unliked and returned with **200**. Sending the same value twice changes nothing the second time.

## In-memory data and restarting the server

All the data lives in the Node process's memory — there is **no database**. Stop the server (Ctrl+C) and start it again: every "Applied", every like and every new post is gone, and the data is back to exactly what `data.js` contains.

Compare with the React side: saved opportunities survive a reload because **your browser** keeps them in `localStorage`. The server's data does not survive a restart because it lives only in memory.

## Commands

Start the server (Terminal 1), from the repository root:

BASH

```
cd server
npm install
npm start
```

You should see `B4F Hub local API running at http://localhost:3001`. Keep it running.

Try a GET request in the browser: http://localhost:3001/api/opportunities

Try requests with curl (on Windows, use Git Bash if JSON quoting fails in PowerShell, or a tool like Postman / Thunder Client):

BASH

```
curl http://localhost:3001/api/posts
curl -X PATCH http://localhost:3001/api/opportunities/101 -H "Content-Type:
application/json" -d '{"applied": true}'
curl -X POST http://localhost:3001/api/posts -H "Content-Type: application/json" -d
'{"content": "Hello B4F!", "category": "community"}'
curl -X PATCH http://localhost:3001/api/posts/1 -H "Content-Type: application/json" -d
'{"liked": true}'
```

Check a file for syntax errors without starting the server:

BASH

```
node --check index.js
```

Run a React client against the server (Terminal 2), as before:

BASH

```
cd client
npm install
npm run dev
```

## Not yet

These are **not** part of today: splitting the server into separate route and controller files, middleware beyond `express.json()`, environment variables, a database, `req.query`, CORS configuration, authentication, NestJS.

## Homework

Read `docs/powerup-session-02/HOMEWORK-EN.pdf`. You will add new endpoints to `server/index.js` yourself, using exactly the patterns in this guide.